

Stitching Hybrid Key Exchange on an In-Order Dual-Issue Cortex-M85: An Honest Measurement Study with Automated Two-Stream Scheduling

Anonymous Author(s)

Abstract

Function stitching—interleaving two independent cryptographic computations into one instruction stream—was proposed by Intel for out-of-order x86 cores. We ask what happens when the idea is carried to a post-quantum/traditional hybrid KEM (X25519 $\times$ the Keccak component of ML-KEM) on an in-order dual-issue embedded core (Cortex-M85). We (i) build an automated pipeline that extends the SLOTHY superoptimizer to two-stream scheduling under carry-flag and register-file constraints; (ii) measure, on silicon, both the pre-registered upper bound of the technique (21–27% per X-Wing operation) and the recovery actually achieved (7–11%), and decompose the gap into its causes by isolating experiments; and (iii) obtain, as an independent by-product, a 4-way MVE-batched Keccak-f1600 that is 18% faster per state than the pqm4 scalar baseline. Our conclusion is honest: on an in-order core, instruction-level stitching of this pair is a real but thin net win, and the thinness is not an accident of tuning but structural—register pressure converts to a fixed yield tax, spill traffic contends for the load/store unit, and store issue blocks scalar co-issue entirely. All claims are backed by cycle-exact measurements on an EK-RA8M1 board with pre-registered decision gates, and the toolchain, harness, and all kernels are published as artifacts.

Keywords. PQ/T hybrid; X-Wing; ML-KEM; X25519; Keccak-f1600; function stitching; superoptimization; SLOTHY; Cortex-M85; MVE.

1 Introduction

The transition answer to quantum-capable adversaries is hybrid key establishment: X-Wing [BCD+24] runs X25519 and ML-KEM-768 and combines their secrets, so that either assumption alone suffices. The price is that both algorithms execute. On desktop-class cores this hybrid tax is noise; on the microcontrollers that anchor embedded TLS stacks it is a deployment argument. Existing mitigations make each primitive faster in isolation, or spend more silicon—a second core [SA25; ESS26]. This paper evaluates a third option on a single core: *hide one algorithm in the issue slots the other wastes*.

The opportunity is microarchitectural. Cortex-M85 is an in-order, dual-issue Armv8.1-M core. X25519 field arithmetic is a long dependency chain of `umaa1` multiply-accumulates and `adds/adcs` carries: we measure IPC 1.03, i.e. 48.7% of the two-issue capacity idles (section 3). The Keccak-f1600 permutation that dominates ML-KEM’s symmetric cost is dense, independent bitwise work (IPC 1.66), and it exercises the ALU/shift resources rather than the multiplier. The two computations are complementary in exactly the sense Intel’s function-stitching white paper [GFG+10] exploited for AES $\times$ SHA-1—but that work targeted out-of-order x86, placed instructions by hand at source level, never measured the idle capacity it claimed to fill, and paired only symmetric primitives.

We transplant the idea and, deliberately, test it to destruction. Concretely:

1. **Automation: a two-stream extension of SLOTHY** (section 4). We extend the constraint-solving superoptimizer [Abd+22] with the missing scalar instruction classes, APSR (carry-flag) lifetimes as solver dependencies, a scalar dual-issue execution model, and a MAC-accumulator forwarding rule—validated on silicon at 17, 156, and 1,184-instruction scale. Fed nothing but the concatenation of two independent streams, the solver discovers the interleaving and reallocates registers *across* streams, beating the best manual stitch at every scale and beating a strong serial baseline built from hand-optimized parts.
2. **A quantified account of why recovery is thin** (section 5). The pre-registered per-operation bound is 21–27%; the end-to-end recovery is 7–11%. We attribute the gap cause by cause with isolating experiments: spill self-cannibalization (a full-ratio kernel yields 3.3%), a structural register-yield tax (+12–14% regardless of how few registers are ceded), LSU contention (58% hiding for a pure MAC chain vs. 27% with spill traffic), and multiplier monopoly (scalar×scalar NTT-style pairing hides −1%). A pairing spectrum experiment pins the co-issue ceiling to one instruction class: `vstrw` blocks scalar issue for both of its cycles, and a three-term model then reproduces the measured stitched time with zero error.
3. **A 4-way MVE-batched Keccak-f1600** (section 6): four permutation states in vector registers, 188.5 cycles per state and round vs. 229 for the pqm4 scalar baseline (−18%), using zero scalar data registers. This is an independently useful result for any Armv8.1-M ML-KEM.
4. **Pre-registered methodology** (section 3). Every go/no-go in this project was written down before measuring: a kill gate after the bound measurement, a 20% bar for the end-to-end phase, and a declared fallback narrative. We report against those commitments, including the parts that failed them.

The paper’s shape is unusual on purpose: the strongest deliverables are a tool, a decomposition of a negative-leaning result, and a by-product kernel. We think this is what an honest evaluation of a plausible idea should look like.

2 Background

2.1 Function stitching and its promise

Intel’s white paper [GFG+10] interleaves two independent computations so that stalls of one are filled by the other, with the ideal $T_{\text{stitch}} \approx \max(T_A, T_B)$: the shorter computation becomes free. The report is source-level and hand-scheduled, gives no measurement of the idle capacity being filled, and relies on out-of-order hardware to absorb scheduling imperfections. One principle we adopt directly: gains must be reported against the *strongest* serial baseline available, including optimizations of the baseline itself found during the work.

2.2 Cortex-M85: the arithmetic of the ceiling

An in-order dual-issue core bounds slot recovery at 50% (Intel’s 4-issue Westmere wasted up to 75%). No hardware reordering exists, so a static schedule’s quality *is* its performance—which cuts both ways: stitching must be scheduled perfectly to work, and a constraint solver, not hand placement, is the natural tool. We pin code in ITCM and data in DTCM (both 0-wait) to remove memory-hierarchy variation.

2.3 X-Wing and its Keccak share

X-Wing [BCD+24] = X25519 + ML-KEM-768 + a SHA3-256 combiner. Key generation runs one X25519 scalar multiplication, encapsulation two, decapsulation one; the ML-KEM sides invoke

Table 1: Stand-alone measurements and slot waste (median cycles).

Kernel	Cycles	Instructions	IPC	Slot waste
X25519 scalar mult (Lenngren)	357,474	366,934	1.03	48.7%
Keccak-f1600 (pqm4)	5,526	9,162	1.66	17.1%
ML-KEM-768 keygen/encaps/decaps	437k/453k/504k	(43/44/44 permutations)		
SHA3-256 combiner (134 B)	7,648	—	—	—

Keccak-f1600 43, 44, and 44 times respectively (measured), plus one combiner permutation. The natural stream pair is therefore X25519 (MAC-bound stream A) $\times$ Keccak (ALU-bound stream B).

2.4 SLOTHY

SLOTHY [Abd+22; AKL25] solves instruction scheduling and register allocation jointly with CP-SAT, for a *single* input stream. Its Armv8.1-M model is MVE-centric: the scalar classes X25519 needs (`umull/umal`, carry arithmetic) were absent, scalar dual issue was not modeled (issue rate 1), and flag lifetimes were not constraints. Those are precisely the extensions of section 4.

3 Pre-Registered Bounds: How Much Is There to Win?

Method. All measurements: EK-RA8M1 (Cortex-M85, 480 MHz), GCC 13.2.1 -O2, code in ITCM, data/stack in DTCM, DWT CYCCNT, $N=100$ medians, 25-cycle harness calibration subtracted. Instruction counts come from Unicorn emulation, cross-checked on two unrelated inputs (equal counts double as a constant-time check, section 7). Emulation is used only for the *numerator*: both kernels are secret-independent straight-line code, so the instruction count is a deterministic constant rather than an estimate, and every cycle figure is measured on silicon. The residual uncertainty is definitional—the emulator charges IT-block bodies individually, which can differ from `INST_RETIRED` by a few percent—so slot-waste figures carry roughly ± 1 percentage point. Functional gates: RFC 7748 KATs [LHT16], SHA3-256 KATs, ML-KEM-768 round trip [Nat24] on pqm4 [Kan+] and a Lenngren X25519 [Len17]—all pass on the board. Before measuring we registered the decision rule: bound $<15\%$ kills the project, $15\% \leq b < 30\%$ forces a reduced two-week pilot, $\geq 30\%$ proceeds.

Table 1 quantifies what Intel never measured: the idle capacity. X25519’s dependency chains leave $2 \cdot 357,474 - 366,934 = 348,014$ empty issue slots per scalar multiplication. Charging Keccak’s instructions against those slots (and capping by stream length) yields table 2: 21–27% per X-Wing operation, average 23.9%. The slot term assumes every idle slot is fillable by the partner stream—optimistic on an in-order core, but section 4 measures IPC 1.99 on a 1:1 interleave, within 0.5% of the two-issue floor, which retrospectively justifies it. Two structural observations: (i) encapsulation is limited by stream length—two scalar multiplications provide more idle slots than Keccak has instructions; (ii) on M85 the Keccak stream is the *shorter* one, so coverage dilution—not slot capacity—sets the bound. This is what our pre-registration anticipated: the plan predicted from M4 accounting [Sim+26] that X25519 would be the longer stream and that coverage would dilute, and the M85 measurement confirms that direction. The gate verdict, mechanically applied: $23.9\% \in [15\%, 30\%) \Rightarrow$ pilot.

Table 2: Pre-registered overlap upper bounds per X-Wing operation.

Operation	Overlappable cycles	Bound
Key generation	209,905	26.4%
Encapsulation	248,670	21.2%
Decapsulation	209,905	24.2%

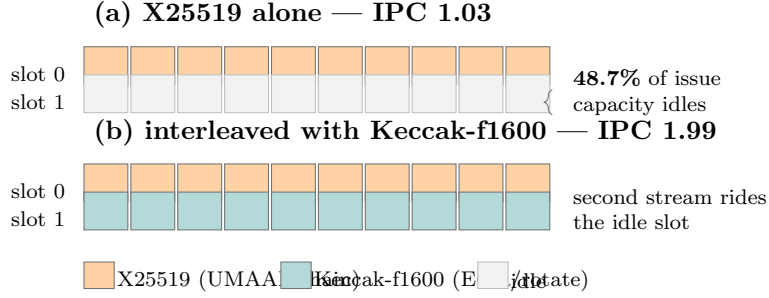


Figure 1: Why stitching should pay on an in-order dual-issue core. The multiply-accumulate chain of a Montgomery ladder step serialises on its own carry dependencies, leaving one of the two issue slots empty in 48.7% of cycles (measured, [table 1](#)). A second, register-disjoint instruction stream can occupy that slot without displacing the first—the microbenchmark of [section 4](#) reaches IPC 1.99, the two-issue floor. The gap between this picture and the 7–11% actually recovered is the subject of this paper.

4 Automation: Two-Stream SLOTHY

4.1 Feasibility by hand first

Microbenchmarks establish that the hardware digests interleaving: a 1:1 mix of a UMAAL chain with a non-flag-setting `eor/rotate` chain runs at IPC 1.99 (within 0.5% of the two-issue floor), hiding 80% of the shorter stream. Making the second stream memory-resident changes the saving by 2 cycles in 4,000: LSU traffic pairs with the MAC stream, so *the stitching-specific spill penalty is zero*. Reserving 4 (even 8) registers for stream B costs a compiled stream A nothing measurable—but that holds only where the compiler still has spill headroom; the hand-tuned fiat multiply pays a 12–14% yield tax instead, which is why the microbenchmark result must not be read as a general one (quantified below). A scripted “zipper” stitcher then produces KAT-validated real kernels; its best manual result, 681 cycles for a 256-bit multiply plus a full Keccak round, is still $1.09\times$ a 626-cycle strongest-serial baseline—and manual sweeps plateaued there. That gap is a register-allocation-plus-scheduling co-optimization problem: the solver’s job description.

4.2 What the solver needed

Our patch (published; upstream model at `241093fa5ddc`) contributes, in order of discovery, each item forced by a measured failure:

- **19 scalar instruction classes** absent from the Armv8.1-M model (`umull`, `umaal`, carry arithmetic incl. immediate forms, `umlal`, shifted/flag variants, ...).
- **APSR lifetimes as dependencies** (`modifiesFlags/dependsOnFlags`): X25519 carry chains survive any legal schedule by construction, mechanizing the one hazard that Intel’s symmetric-only pairing let them ignore.
- **Scalar dual issue**: issue rate $1 \rightarrow 2$ plus a second scalar ALU slot (MAC and LSU unchanged).

Table 3: Serial vs. best manual stitch vs. solver, board-measured (cycles/iteration, correctness-checked against C references).

Scale	Serial	Manual	SLOTHY	Solver vs. manual
17 instr (concept)	13.06	—	11.07	(−15% vs. serial)
156 instr	110.06	88.07	85.06	−3.4%
1,184 instr (full kernel)	727.0	681	657.0	−3.5%
1,130 instr (flat carry-mul)	702.0	—	630.0	(−10.3% vs. serial)
1,110 instr (bit-interleaved)	636.0	—	579.0	(−9.0% vs. serial)
Co-issue 1,222 instr (scalar×MVE)	1,365	1,018	996	−2.2%

- **MAC accumulator forwarding** (model v0.3): silicon executes a dependent `umaal` chain at 1 cycle/instruction through the accumulator operand; the stock 2-cycle latency mis-priced stream A by $2\times$ and made the solver *lose* to the zipper (1,319 vs. 1,018 cycles) until fixed—after which it wins (996). Model accuracy is not a nicety; on an in-order core it decides the outcome.
- **Three safe-transform rules** for feeding compiler output to the solver, each validated by a corruption we debugged to zero: split doubleword accesses (the model treats `ldrd/strd` as 4-byte), never reuse a spill slot (WAR/WAW memory anti-dependencies are not preserved), and order base-destroying load pairs last. Reported upstream as three issues.

Input needs no annotation: the two streams are concatenated (or pre-zipped, see below) and dependency analysis discovers their independence.

4.3 Validation ladder

Table 3 is the tool’s report card. Three points. First, the solver beats the best manual stitch wherever one exists, and at full scale the manual technique *cannot even be constructed*: the zipper’s premise—a static register partition between streams—collapses when stream A uses all registers freely (we demonstrated the resulting live-value corruption as a hard fault); the solver, allocating globally across streams, has no such premise. Second, the fidelity ladder ends at real material: a fiat-crypto field multiplication (BoringSSL’s carry-mul) against a pqm4-representation (bit-interleaved) Keccak round reaches 579 cycles—9% under serial and **7.5% under the 626-cycle strongest-serial baseline assembled from hand-optimized parts**, even though stream A is compiler output. The manual-era gap is closed by automation. Third, model fidelity: prediction error is $<1\%$ at small scale, 5–7% at full scale (split-window boundary stalls), and 12% for scalar×MVE after the v0.4 store fix (section 5). The evolution of one work unit (one multiply + one round) summarizes the project: $875 \rightarrow 799 \rightarrow 681 \rightarrow 657 \rightarrow 630 \rightarrow \mathbf{579}$ cycles, −34% from the first zipper.

The practical pipeline that emerged: *write the real arithmetic in C, reserve stream B’s registers with `-ffixed` (the compiler inserts correct spills), and let the solver schedule*. Hand-written assembly that saturates all 14 registers cannot be stitched by *any* scheduler without adding spills, which a scheduler may not do.

5 Why Recovery Is Thin: A Decomposition

The bound said 21–27%; complete-ratio kernels deliver 7–11% (section 6). This section is the paper’s core: each cause isolated by an experiment designed to show only it.

(a) **The scalar plan self-cannibalizes.** Stitching at the real X-Wing ratio (four multiplies per round, 2,526 instructions) yields $1,573 \rightarrow 1,521$ cycles: **3.3%**, an order below the bound. The

Table 4: Causes of the bound-to-actual gap, each isolated on silicon.

Cause	Isolating experiment	Result
(a) Spill self-cannibalization	full-ratio fiat kernel (2,526 instr)	bound 21%+ $\rightarrow$ 3.3%
(b) Register-yield tax	-ffixed 2 vs. 3 registers	+12.1–13.5% , count-independent
(c) LSU contention	real mul (spills) vs. pure MAC chain	hiding 27% vs. 58%
(d) Multiplier monopoly	scalar $\times$ scalar NTT-style	hiding -1%
(e) Pipe separation (control)	scalar $\times$ MVE NTT-style	hiding 62%

mechanism: to cede registers to stream B, stream A (compiled with `-ffixed`) spills; the spill loads/stores then occupy the very LSU slots stream B’s memory-resident state needs. The idle capacity is real, but filling it consumes it.

(b) The yield tax is structural. Ceding 3 registers costs the fiat multiply $+13.5\%$ ($287.6 \rightarrow 326.5$); ceding only 2 costs $+12.1\%$. The near-equality is the finding: the tax buys the *existence* of a second live register set—the compiler restructures live ranges—and is nearly flat in the count. (Contrast the microbenchmark’s free reservation in [section 4](#): that stream had slack; real carry-saturated code does not.)

(c) LSU contention, quantified. A pure `umaal` chain hides 58% of an MVE Keccak stream; the real multiply with its spill traffic hides 27%. The difference is load/store-unit competition between A’s spills and B’s state accesses.

(d–e) The pairing rule. Scalar $\times$ scalar NTT-style pairing (both streams multiplier-bound) hides -1% : the MAC pipe is a monopoly, and this negative control validates that our gains elsewhere come from complementarity, not measurement art. Moving stream B to MVE’s separate multiplier flips the same experiment to 62% hiding—the embedded, in-order, measured version of Intel’s qualitative claim that mixed register-file pairs are the easy case.

The co-issue ceiling has one owner. For the scalar $\times$ MVE plan, hiding saturates at 60%. A pairing-spectrum experiment (one MVE class at a time against the `umaal` chain) convicts a single class: `veor`, `vldrw`, and `vshl/vsri` all pair at 99%—`vldrw` is a 2-cycle instruction that still admits one scalar per cycle throughout—but `vstrw` admits none in either of its two cycles (-1% hiding). At the real mix this predicts the measured stitched time exactly: $996 = 611$ (scalar) $+ 242$ (store block: 121×2) $+ 143$ (B-internal stalls, into which scalar issue also cannot proceed)—**zero error**. Encoding the rule in the model (store occupies STORE+SCALAR; v0.4) cut prediction error from 27% to 12%; honesty requires noting the v0.4-rescheduled kernel did not beat the v0.3 schedule on silicon (1,018 vs. 996). Below the 853-cycle sub-ceiling (B-stall removal), progress belongs to design—fewer stores—not scheduling.

6 Plan B Results: Scalar $\times$ MVE, End to End

Given (a)–(c), the register-file-separated plan is the deployable one: X25519 stays scalar with all 14 GP registers; Keccak-f1600 moves to MVE.

4-way batched MVE Keccak-f1600. Armv8.1-M MVE has eight 128-bit vector registers: exactly four 32-bit-interleaved Keccak states per lane position. Our generator lays out the permutation with uniformized rotations across lanes and anchors state addressing so that base-register count shrinks $5 \rightarrow 3 \rightarrow 2$ at zero cost (`vldrw` ± 508 offsets). Result: **188.5 cycles per**

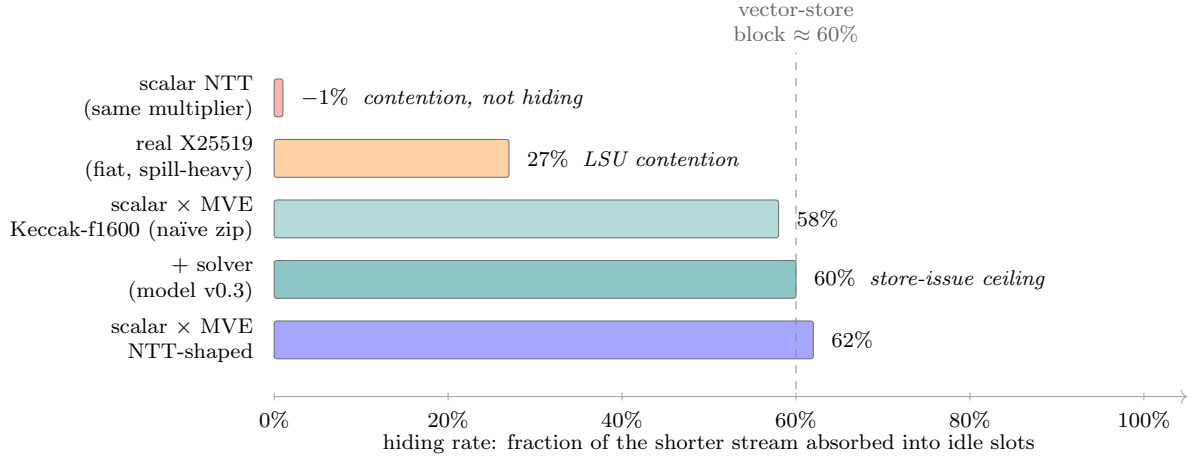


Figure 2: Hiding rate by pairing and pipe. Two streams that share an execution resource do not stitch at all (scalar NTT against the UMAAL chain: -1% , a multiplier tug-of-war); a real, spill-heavy X25519 pairs poorly because its spill traffic contends with the partner on the load/store unit (27%); moving the partner to a different pipe (MVE) recovers most of the slot ($58\text{--}62\%$). The residual is not a scheduling failure but a structural one: vector stores occupy the scalar issue slot, and $121 \text{ stores} \times 2 \text{ cycles}$ account for the exposure exactly (section 5).

Table 5: X-Wing operations at real ratios: measured units, projected operations, and controls.

	Serial	Stitched	Saving
Unit 4:1 (keygen/decaps ratio)	2,008	1,772	-11.8%
Unit 8:1 (encaps ratio)	3,256	3,010	-7.6%
Key generation (projected)	544k	482k	11.4%
Encapsulation (projected)	886k	821k	7.3%
Decapsulation (projected)	(4:1, as keygen)		$\sim 11.4\%$
Control: coarse alternation (3:3 unit)	3,220	3,199	$-0.7\% \approx 0$
Same unit, instruction-level stitch	3,220	2,943	-8.6%

state and round vs. 229 for pqm4 scalar (-18%), with zero scalar *data* registers—the entire GP file is left to X25519. Lane transposition glue costs 2.5% (measured); ML-KEM-768’s 43–44 permutations per operation fill the 4-way queue naturally, and the X-Wing combiner’s single SHA3-256 permutation rides the same queue as the 45th entry: no extra mechanism, cost $\leq$ one batch round.

End-to-end at real ratios. We measure macro-units at the true operation mix (fiat multiply : batched round = 8:1 for encapsulation, 4:1 for keygen/decapsulation; real instruction streams, flash-resident at realistic footprint, correctness-verified) and project by unit counts:

Two control results matter. Coarse alternation—switching algorithms at function granularity, the “easy” software answer—recovers nothing (-0.7%): the gain genuinely lives at instruction granularity. And the balance law is visible: 4:1 units save more than 8:1 (11.8% vs. 7.6%), because saving is bounded by the hideable stream B share; the 1:1 unit sits at 8.1% (partially hidden).

Verdict against the pre-registered bar. We had committed to a 20% end-to-end bar for the aggressive narrative. $7\text{--}11\%$ misses it. Per the pre-registered fallback, this paper reports the automation, the decomposition, and the MVE kernel as the contributions, and the thin-but-real

end-to-end win as the measured answer to the Stage-4 question (“does the gain survive whole operations?”—yes, at 7–11%).

7 Constant Time

Stitching adds no timing channel, by construction and by measurement. *Structurally*: the transform is compile-time only—it reorders instructions, renames registers, and (via `-ffixed` compilation) inserts spills to fixed stack/DTCM offsets; it inserts zero branches and no secret-dependent addressing. Both input streams are constant-time to begin with (fiat-crypto’s generation guarantee; Lenngren’s masked cswap ladder; Keccak is a fixed permutation; the 4-way queue length is the per-operation constant 43–45). On M85, integer and MVE latencies are operand-independent and ITCM/DTCM are 0-wait, so a fixed instruction sequence implies fixed cycles. *Empirically*: the stitched fiat×MVE kernel measured on two unrelated input sets gives median $\Delta = 0$ cycles (979,065 vs. 979,065 at ×1000 loop resolution), and Unicorn instruction counts agree on two inputs for every kernel. We did not run dueduct-style statistics: with no physical path for value-dependent timing in this configuration, the two-input check is confirmation, not evidence of first resort. Power side channels are out of scope.

8 Related Work

Intel’s stitching [GFG+10] is the origin; it is hand-placed, unmeasured on the waste side, symmetric-only, and out-of-order-x86-only. SLOTHY [Abd+22; AKL25] superoptimizes single streams; our extension makes the two-stream problem—including inter-stream register reallocation, which no fixed-partition manual technique can express—a solver problem. Becker–Kannwischer’s scalar/vector Keccak [BK22] pioneered mixed-register-file execution on Cortex-A; our pairing-spectrum experiment quantifies exactly when that works on in-order M-class (loads pair, stores block). Multi-core hybrid parallelization [SA25; ESS26] is orthogonal: it buys overlap with silicon, we recover it from issue slots. Hybrid cost accounting on M4 [Sim+26] anchors our baselines and supplied the stream-balance estimate we pre-registered; the M85 measurement confirms its direction, with X25519 the longer stream and coverage dilution—not slot capacity—setting the bound. ML-KEM implementation baselines: pqm4 [Kan+], FIPS 203 [Nat24].

9 Conclusion

On an in-order dual-issue core, function stitching of X25519 and Keccak is real, automatable, and thin: 7–11% end to end against a 21–27% bound, with every point of the gap accounted for by an isolating experiment. The honest summary for practitioners: (i) if you can spend a vector unit, spend it on batching (our 4-way MVE Keccak is 18% faster per state before any stitching) and take stitching’s 7–11% on top where the ratio is favorable; (ii) do not hand-schedule—model accuracy decides wins (a single mis-priced latency flipped our solver from losing by 30% to winning), and the extended solver plus the C-`-ffixed`-solve pipeline is reusable; (iii) on cores without reorder hardware, the pairing table (which instruction classes co-issue) is the design document. The tools, harness, kernels, KATs, and pre-registration documents ship as artifacts.

Limitations. Macro-unit fidelity (ladder control flow and absorb/squeeze glue, bounded at 2.5%, sit outside the measured units); single platform (Cortex-M55 untested—[TODO: M55 board]); the scalar dual-issue model remains TRM-unvalidated in its pairing details beyond the rules measured here.

Energy is deliberately out of scope. This is a scheduling study on a fixed-clock in-order core: every claim we make is a cycle count, and the quantity we optimise—issue slots recovered by interleaving two instruction streams—maps to energy only through the same clock. We therefore report cycles throughout and make no joule-per-operation claim. The one place where this could hide something is the MVE path of Plan B, since vector units carry a different power profile than the scalar pipeline they displace; a scalar vs. vector energy comparison at equal cycle count is future work requiring power instrumentation, and would refine—but not overturn—the slot-recovery accounting presented here.

References

- [Abd+22] A. Abdulrahman, H. Becker, M. J. Kannwischer, and F. Klein. *Fast and Clean: Auditable High-Performance Assembly via Constraint Solving*. IACR ePrint 2022/1303. 2022.
- [AKL25] A. Abdulrahman, M. J. Kannwischer, and T.-H. Lim. “Enabling Microarchitectural Agility: Taking ML-KEM and ML-DSA from Cortex-M4 to M7 with SLOTHY”. In: *AsiaCCS*. 2025.
- [BCD+24] M. Barbosa, D. Connolly, J. D. Duarte, et al. “X-Wing: The Hybrid KEM You’ve Been Looking For”. In: *IACR Communications in Cryptology 1.1* (2024). ePrint 2024/039.
- [BK22] H. Becker and M. J. Kannwischer. *Hybrid Scalar/Vector Implementations of Keccak and SPHINCS+ on AArch64*. IACR ePrint 2022/1243. 2022.
- [ESS26] S.-W. Eum, M.-H. Song, and H.-J. Seo. *Performance Analysis of a Thread Pool-Based Parallel Execution Model for Hybrid Post-Quantum TLS 1.3 Handshakes*. IACR ePrint 2026/440. 2026.
- [GFG+10] V. Gopal, W. Feghali, J. Guilford, et al. *Fast Cryptographic Computation on Intel Architecture Processors Via Function Stitching*. Tech. rep. 323686. Intel, 2010.
- [Kan+] M. J. Kannwischer, R. Petri, J. Rijneveld, P. Schwabe, and K. Stoffelen. *pqm4: Testing and Benchmarking NIST PQC on ARM Cortex-M4*. <https://github.com/mupq/pqm4>. commit cc2c1b992b60.
- [Len17] E. Lenngren. *X25519 for ARM Cortex-M4*. <https://github.com/Emill/X25519-Cortex-M4>. 2017.
- [LHT16] A. Langley, M. Hamburg, and S. Turner. *Elliptic Curves for Security*. Tech. rep. RFC 7748. IETF, 2016.
- [Nat24] National Institute of Standards and Technology. *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Tech. rep. FIPS 203. NIST, 2024.
- [SA25] F. Segatz and M. I. Al Hafiz. *Efficient Implementation of CRYSTALS-KYBER Key Encapsulation Mechanism on ESP32*. arXiv:2503.10207. 2025.
- [Sim+26] M. Sim, M. Lee, S. Cho, Y. Hyoun, and H. Seo. *Evaluating Hybrid KEM/DSA for KpqC and NIST PQC on ARM Cortex-M4*. IACR ePrint 2026/1414. 2026.

A Measurement Details

EK-RA8M1, 480 MHz, FSP BSP clocks; code ITCM, data DTCM; DWT CYCCNT medians of $N=100$, 25-cycle calibration; Unicorn instruction counts (IT blocks counted individually; may differ from INST_RETIRED by a few percent). PMU event counters were nonfunctional on our board specimen; cycle+emulated-count arithmetic replaces front/back-end stall attribution.

Version pins: pqm4 cc2c1b992b60, upstream SLOTHY 241093fa5ddc (+ our patch v0.1–v0.4), Lenngren X25519 vendored with license. The harness auto-verifies every kernel (KAT/reference match) before timing; a constant-time PASS/FAIL line accompanies each harvest.

B Reproduction Hazards

Loop bodies need `.balign 16` (± 1 cycle/iteration otherwise); large ITCM functions must use `movw/movt` instead of literal pools; DTCM stack switches must also move `MSPLIM`; `CYCCNT` writes are ignored while the PMU is disabled; the vendor IDE may silently shrink the TrustZone flash partition below the firmware size (fix via the partition CLI). Doubleword accesses must be split and spill slots never reused when feeding compiler output to the solver ([section 4](#)).